

CS482/495/496 Software Project Proposal: Football Team Management

Nyrique' Butler, Andrew Dominquez, Leslie Kim, Chloe Miranda, Jack Wehberg

2025-10-24

1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

- Client name: Dr. Cui
- Client title: stockholder
- Client email address: lcui@loyola.edu
- Client employer: Kids' Activity Management Organization
- How you know the client: Our client, Dr. Cui, is acting as a stockholder for a teens soccer league.

2 Project Description

2.1 Overview

[Add a few paragraphs describing your project succinctly. What problem are you trying to solve, what is the purpose of your project? Why does your client want this project?]

Our client, Dr. Cui is a main stockholder of Kids' Activity Management Organization. He is requesting a web application specifically for a local soccer league to allow league administrators to manage seasons and for the adults registered in the league to have a place to view game schedules for the whole season. This will also be used as a platform for registered adults in the league to connect with each other online as a community.

2.2 Key Features

[At this point you should have a basic understanding of your client's needs. List out the key features of the software system the client wants you to build.]

Key features:

- Login
- calendar of matches that spans for the season
- match generator at start of season
- match manager as season progresses into playoffs

social media

- picture/video feed
- commenting on posts
- liking posts

2.3 Why this Project is Interesting

This app centralizes access to the league not only for registered parents but for prospective parents who are looking to enroll their kids in an activity. This app is also enticing because it uses database management and a social media style interface on its main page. For example, user accounts are stored in the database, the admin can update the scoreboard and playoff bracket, and registered adults can post comments, videos, or images on the feed.

Additionally, this project is meaningful because it makes it easier for parents and their kids to stay informed about matches, team performance, and schedule. It is also interactive for registered parents to share media and engage with others. The app supports the league community by providing an online space to connect and stay informed throughout the season.

2.4 Areas of CS required

The most relevant subfields are web development, database management, and security.

2.5 Potential Concerns and Questions

All of us have limited to no experience in web programming. We anticipate that implementing the back-end with MongoDB and Node.js will be the most difficult part. So we might have questions on how to set up the brackets or the scoreboard later on in this project.

3 Requirements

3.1 Non-Functional Requirements

ID	NFR Title	Category	Description
NFR1	Blue theme	Usability	Website should have a blue color theme
NFR2	Include logo	Usability	A custom logo should be present in the website
NFR3	Color-coded calendar	Usability	The calendar should have color-coded dates
NFR4	Post with minor protection	Security	All posted content with minors should be blurred for un-registered users
NFR5	Password security	Security	Accounts should be encouraged to have strong password
NFR6	Admin control	Security	Only the admin should handle any sort of live data

Table 1: Non-Functional requirements

3.2 Functional Requirements (User Stories)

[In CS482, all functional requirements are written as User Stories. In CS496, some projects may use a different template to write the requirements. The table below is an example of writing the Stories. Adapt accordingly to different templates or if you want to display more info.]

ID	Story Title	Points	Description
S1	Story Example 1	5	As a user, I want to write a user story example, so that people will understand them.
S2	Story Example 2	2	As a user, I want to write a user story example, so that people will understand them.

Table 2: Functional requirements as User Stories.

4 System Design

4.1 Architecture

Our team is following the MVC architecture. In our design, the “View” layer would handle the user interface with HTML, CSS, and JavaScript. The “Model” layer would handle the database using MongoDB. The “Controller” layer would manage incoming requests by using Node.js features. This includes applying user account permission and account authentication. We will use the Express module to handle requests.

4.2 Diagrams

[CS482, on sprints/iterations 2-3, you need to create and update a diagram (check the assignment for which type of diagram). On CS496, since before sprint/iteration 1 you should have a class diagram and keep it up-to-date.]

4.3 Technology

The programming language our team will be using is JavaScript, along with HTML & CSS for webpage development. JavaScript allows us to use the same language for both client and server side, making development more efficient. We will be using Node.js as our main framework as it allows us to use JavaScript on the server side as well. Our team will be using MongoDB as our database because it integrates easily with JavaScript. To test our code, we will be using Jest as it is simple to write & run tests, helping us to meet our coverage goal.

4.4 Coding Standards

The team will follow consistent coding standards to ensure readability, maintainability, and reliability throughout development. Only working code will be committed to the repository, and each commit message must clearly describe what was changed or added. Code will be pushed in small increments to make testing and debugging easier, and any commented-out or developer-only notes will be removed before release. All variable declarations will use `const` or `let`, instead of `var`, and complex methods will include inline comments explaining their purpose. File names will follow the lowercase-hyphen format, variables and functions will use camelCase, and constants will be written in `UPPERCASE_WITH_UNDERSCORES`. Test names will clearly describe the function being tested and the expected outcome.

Functions will be kept short, serving a single clear purpose, and each will include JSDoc-style comments explaining its parameters and return values. Code formatting will follow a consistent style, including two-space indentation, single quotes for strings (unless otherwise necessary), one blank line between functions, and removal of unused variables or imports. Testing will be conducted using Jest, with at least 60% code coverage required for commits and a final goal of 80% coverage at project completion. Project documentation will be maintained alongside code updates to ensure accuracy and alignment with the current implementation.

4.5 Data

Here is a description of the entities that will be stored in our database for the football league.

The football league is comprised of a league administrator, teams, team managers, registered adults and their registered kids. A league administrator or “admin” has a unique id and their name, username, email, phone number, date of birth and login password stored. A league is comprised of teams. A team has a unique id, a team manager, team name, and a roster of registered kids. Team managers have a unique id, their name, username, email, phone, date of birth, login password and their team’s id. Registered kids are registered by their parents and they input their children’s name, and date of birth. A kid cannot register itself. Parents/guardians of kids must be registered adults. A registered adult has their name, username, email, phone number, date of birth and login password stored as well as the id of their registered kid.

In the league, there is a season for football. A season has a unique id, a start date, an end date and an eventual champion. Seasons are comprised of games. A game has a unique id, date, start time, end time, place, weather, status in the season (normal, playoff, championship), a winning team and a losing team.

Live chats are recorded on the landing page during games and are comprised of messages. Messages have a unique ID, a username from registered adult, date and time.

resulting tables

- League_Admin(@id_admin, name, email, phone, date_of_birth, password, username)
- Team_Manager(@id_manager, name, email, phone, date_of_birth, password, #id_team, username)
- Registered_Adult(@id_adult, name, email, phone, date_of_birth, password, username)
- Registered_Kid(@id_kid, name, date_of_birth, #id_adult, #id_team)
- Team(@id_team, name_team, #id_manager, wins, losses)
- Season(@id_season, start_date, end_date, #id_champion_team)
- Match(@id_game, date, start_time, end_time, final_score, place, weather, status, #id_team1, #id_team2, #id_season)

4.6 UI Mocks

The entire sketch so far is hypothetical and is subject to change upon testing or further discussion as to what should be prioritized upon first loading the website. When the page is loaded, a guest/user/admin will see the home page that has a feed centered in the center with an active standings tab to the left, a scaled down schedule display on the right and another widget just below that. There would be a static bar at the top that will prompt users to log in at the top right. Once a user presses log in, there would be an option to enter your email and password. Otherwise, a guest will create a new account via a sign up button that will appear at the bottom of the card. The sign up card would ask for a whole bunch of information that will serve as an identifier to ensure not just anybody can sign up for the website. It was originally planned for the administrator dashboard to be its on page, but upon discussion it was decided that the administrator dashboard would be appear on the profile of the admin and be hidden on all non-administrator accounts.

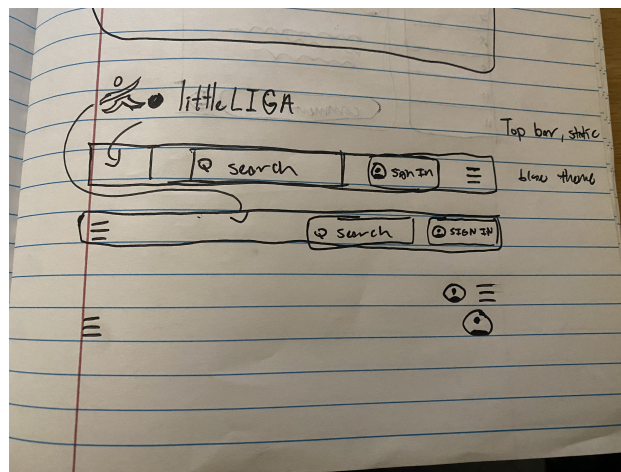


Figure 1: Static Bar

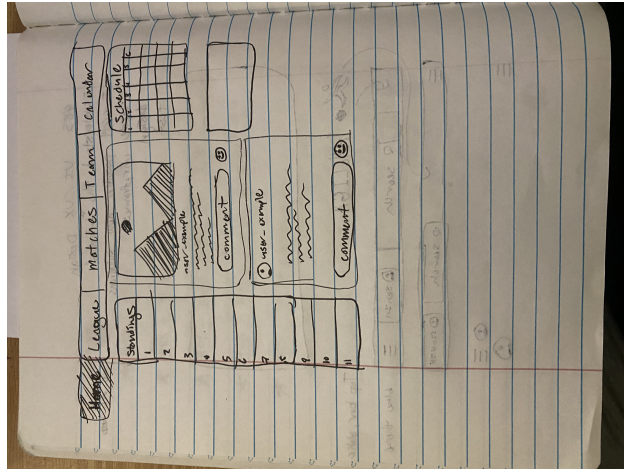


Figure 2: Main landing page

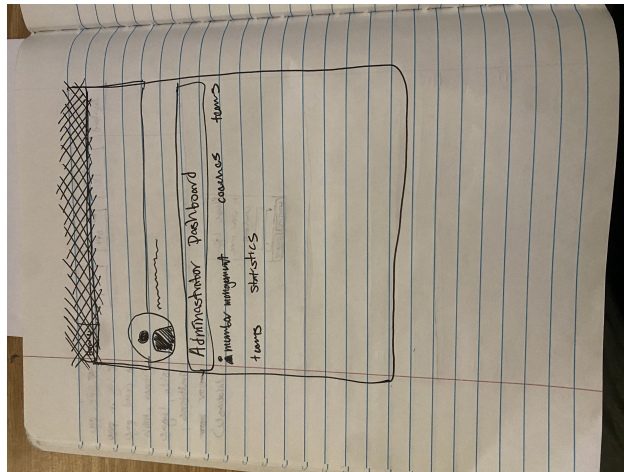


Figure 3: very rough Admin dashboard on profile page

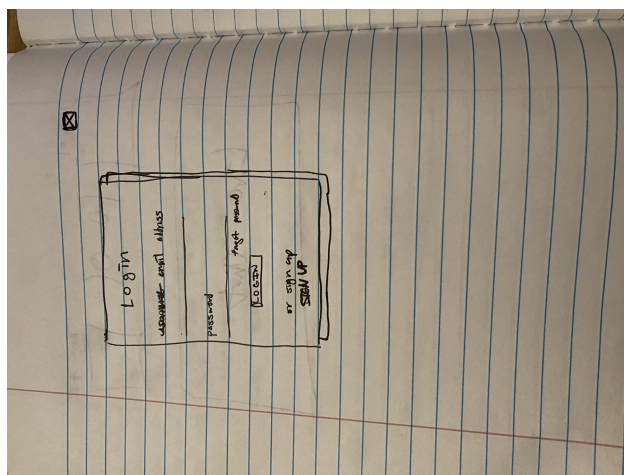


Figure 4: part 1 of log in page(login)

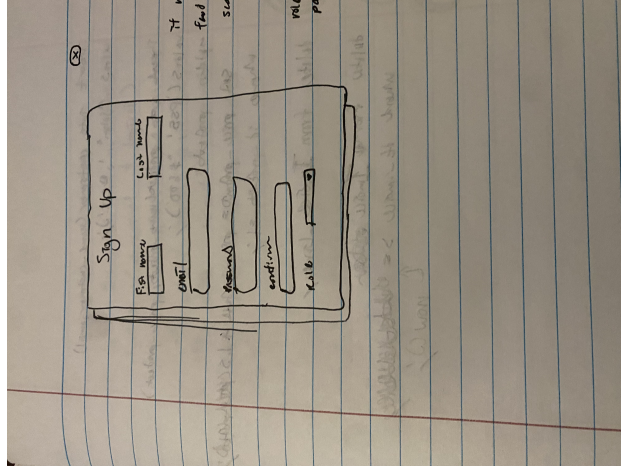


Figure 5: part 2 of log in page(sign up)

5 Iterations

5.1 Iteration Planning

Iteration	Dates	Stories	Points
1	10/09 - 10/23	G34 Account Creation, E35 User Login/Logout, L36 Admin Dashboard, L7 Minor reassignment, L27 Approving adults, L9 Calendar display, L29 adding/deleting matches from calendar, M6 Managing the team, M3 creating a new team, M5 inviting minors to teams, L30 creating randomized bracket at start of season, L31 automatic bracket generation throughout season, L32 playoff bracket generation, L33 Championship/season ending, E21 scoreboard (live), E22 future matches, E23 past matches, E26 weather	24
2	02/01 - 03/01	S3 Story Title, S4 Story Title, S5 Story Title, S6 Story Title	17
3	03/01 - 04/01	S7 Story Title, S8 Story Title, S9 Story Title, S10 Story Title, S11 Story Title	21
4	04/01 - 05/01	S12 Story Title, S13 Story Title, S14 Story Title, S15 Story Title	19
5	05/01 - 06/01	S16 Story Title, S17 Story Title	06
Total:			70

Table 3: Iteration Planning for Incremental Deliveries

5.2 Iteration/Sprint 1

5.2.1 Planning

The stories we initially planned for this iteration are listed on the table at the Iteration 1 row, excluding G34 Account Creation, E35 User Login/Logout, and L36 Admin Dashboard. Our focus for the first iteration is to choose some user stories from most of the major features, which are admin, manager, calendar, bracket

generation, and match data, to get started. It was decided that Chloe would do the calendar, Leslie would do admin, Jack would do manager, Andrew would do bracket generation, and Nyrique would do match data features.

At the beginning, we planned for everyone to do four story points. But we forgot about account features and the admin dashboard, which are prerequisites to most of our planned user stories. So, G34 Account Creation, E35 User Login/Logout, and L36 Admin Dashboard were added after planning and during the iteration because they were high priority for the admin feature user stories and setting up the website. Because of it, one member ended up doing more story points than others unintentionally.

5.2.2 Work Done

The stories we completed are G34 Account Creation, E35 User Login/Logout, L36 Admin Dashboard, L7 Minor reassignment, L27 Approving adults, L9 Calendar display, L29 adding and deleting matches from the calendar, M5 inviting minors.

The stories we partially completed are E21 live scoreboard, E22 future matches, E23 past matches, E26 weather. M3 Team creation, M6 managing a team, L30 season start bracket, L31 automated match decisions, L32 playoff generation, L33 Championship

Chloe worked on L9 Calendar display and L29 adding/deleting matches from calendar. Leslie worked on G34 Account Creation, E35 User Login/Logout, L36 Admin Dashboard, L7 Minor reassignment and L27 Approving adults. Jack worked on M6 Managing the team, M3 creating a new team, and M5 inviting minors to teams. Andrew worked on L30 creating randomized bracket at start of season, L31 automatic bracket generation throughout season, L32 playoff bracket generation, and L33 Championship/season ending. Nyrique worked on E21 scoreboard (live), E22 future matches, E23 past matches, and E26 weather.

5.2.3 Testing Coverage

File	% Stats	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	98.32	76.92	95.74	98.55	
FootballTeamManagement	100	100	100	100	
calendar-config.js	100	100	100	100	
FootballTeamManagement/controller	98.16	100	100	98.16	
MinorController.js	98.47	100	100	98.47	28,42
UserController.js	98	100	100	98	17,48,85,99
FootballTeamManagement/model	87.23	50	94.11	88.04	
DbConnection.js	98.9	50	100	100	10-11
MatchDb.js	100	100	100	100	
MinorDb.js	100	100	100	100	
SeasonScheduleDao.js	75	50	85.71	75	46-47,128,128,144,157-164
TeamDao.js	100	100	100	100	
UserDao.js	100	100	100	100	
FootballTeamManagement/util	100	100	100	100	
PasswordUtil.js	100	100	100	100	

Figure 6: Iteration 1 coverage report

This coverage is good enough because all tests statement coverage are above 80%. We tested most parts of the code for the Controller and the Model. We didn't test the "catch (err)" block in the try-catch that most functions have because the coverage already shows good enough statement coverage and it seems redundant to test if it catches the error properly. For Calendar's testing, Chloe got some weird edge cases like leap years but there may be more found later, and coverage may not show this.

5.2.4 Retroespective & Reflection

The first challenge in this iteration was setting up the programming environment since it was originally confusing to understand how to connect MongoDB to the program and the purpose of the App.js and Server.js from the example code. After referencing some tutorials and somewhat referring to "ParkingLotExampleJS", we learned how to set up everything properly.

Also related to the programming environment, there was an issue in MongoDB database. Before the fix, the database would clear the data after a couple hours. This was an inconvenience when testing the code manually, especially testing admin features. The solution was to not use the default database, test, by creating a new database on the website and edit the ".env" file to include that database's name.

On the topic of databases, the user stories we worked on for this iteration had some things that relied on others, and because we worked on those components separately, we had to ensure that we knew how data was going to be sent to the server.

Another major challenge was creating the Jest test code for the Controller Javascript files. It was extremely confusing at first because the entire example tests for the Controller in “ParkingLotExampleJS” didn’t make sense. But it became clearer once we researched about Jest mock functions and noticed how the Jest test code lines corresponded to the code being tested.

Styling the frontend was also challenging. For example, we wanted to use a Bootstrap modal for team ID input in minor reassignment. Implementing the modal didn’t go as planned because we couldn’t get it to work. In the end, using `prompt()` was sufficient, even though it was not as visually appealing as a modal.

Lastly, there was an issue in planning, which we discovered during the iteration, because we forgot to include user stories for account features and admin dashboard in our spreadsheet, which are connected to most planned stories in this first iteration.

To summarize, in this iteration, we learned how to set up MongoDB and use Node.js with Express to implement the Controller and Model. It was also our first time working with backend, which helped us understand how the database interacts with the server through routes.

For our next iteration, it will probably be best to have better planning for our user stories so that they are a little bit more unified and make an order that shows logical progress rather than grab at available points.

5.3 Iteration/Sprint 2

5.3.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.3.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.3.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.3.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.4 Iteration/Sprint 3

5.4.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.4.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.4.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.4.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.5 Iteration/Sprint 4

[CS496 has 5 sprints. CS482 only has only 3 sprints (remove Iterations 4 and 5 from this doc if you are writing a doc for 482)]

5.5.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.5.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.5.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.5.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.6 Iteration/Sprint 5

5.6.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.6.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.6.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.6.4 Retroerspective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

6 Final Remarks

6.1 Overall Progress

[Have you completed everything? If so, present evidence on how you brought value to your client, and the overall client satisfaction. Otherwise, estimate how much progress you done and how long it would take to finish this project.]

6.2 Project Reflection

[Your personal reflection on the project. What lessons did you learned. What would you have done differently. How can you do better work in future projects? You may write this as a team or per person (or both)]

Appendix

[Appendix section if needed]